1. Bootstrap: from a fresh machine to a first build

`v001/bootstrap.sh` takes a Linux machine from nothing to a built and tested TreeSync. It is safe to run again at any time: whatever is already there is left alone.

1.1 What you need

- **Linux**: RHEL 9 and its rebuilds (Rocky, Alma), Fedora, or Linux Mint / Ubuntu / Debian. TreeSync is Linux only for now.
- **A user that can use `sudo`**, for installing packages (bootstrap shows every command and asks before it runs any).
- **Network access**, to install packages and, if U++ is not on the machine yet, to download it.
- About **2 GB of disk** for U++ and the build output.

1.2 Get the code

```
git clone https://github.com/JayACarlsonAtHome/jac317.git
cd jac317/v001             # on dev-work, the default branch
```

`dev-work` is where the work happens: it is tested on every push, but it is the newest code, not a release. With an SSH key on GitHub, `git@github.com:JayACarlsonAtHome/jac317.git` works too.

Everything below runs from `jac317/v001`.

1.3 See what bootstrap would do (optional)

```
./bootstrap.sh --dry-run
```

This checks the machine and prints the plan: which requirements are present, the install command for each missing one, and whether U++ would be fetched. Nothing is installed or built.

1.4 Run bootstrap

```
./bootstrap.sh
```

It works in five steps.

Step 1: the Linux family

It reads `/etc/os-release` and picks the install recipes for `rhel`, `fedora` or `linuxmint` (Ubuntu and Debian count as `linuxmint`).

Step 2: the requirements

Each requirement in `setup/recipes.conf` is checked and listed as `ok` or `MISSING`:

Name	What it is	Needed for
`gcc`	GCC 15 or newer (RHEL: `gcc-toolset-15`)	building (required)
`clang`	Clang 21 or newer	building (required)
`gui`	GTK 3 and the libraries U++ GUIs build with, OpenSSL for SSH	building (required)
`tools`	make, git, curl, tar, xz	fetching and building U++ (required)
`uppfix`	A one-line fix to U++'s heap (gcc release builds crash without it)	building (required)
`testing`	bc and OpenSSH client and server	the compiler comparison and the SFTP tests (optional)
`checks`	AddressSanitizer + UBSan runtimes and valgrind	the checked test runs (optional)

For the missing ones it shows the install commands, for example on RHEL:

```
To install (these change the system; most need sudo):
  gcc: sudo dnf install -y gcc-toolset-15
  gui: sudo dnf install -y gtk3-devel libnotify-devel ...

Run these now? [y/N]
```

Answer `y` to run them (you will be asked for your `sudo` password), or `n` to stop and run them by hand. If a required one is still missing afterwards, bootstrap stops here.

Step 3: U++

TreeSync is built with U++ (its build tool `umk` and its sources `uppsrc`). Bootstrap looks for one in this order:

1. `$UPP_DIR` (if you set it)
2. `~/upp_basefix`: an install made by upp_basefix, U++ pinned to a GitHub commit plus fixes (0 warnings with gcc and clang, sanitizer and valgrind clean, the heap fix below included)
3. The shared copy on the GFS disk: `/run/media/jay/GFS/upp` (or `/media/jay/GFS`, `/mnt/GFS`) with `upp-bin/rhel/umk` or `upp-bin/mint/umk`
4. `~/u++`

Each is taken as a pair (its `umk` with its own `uppsrc`). If none is found it offers to install U++ through upp_basefix into `~/upp_basefix`: it clones upp_basefix, runs its bootstrap (the pinned U++ from GitHub, `umk` built from it, the fixes; it lists the system packages U++ needs and installs them only when you say yes, with `sudo`), then `upp_basefix.sh install`. To put it somewhere else:

```
./bootstrap.sh --upp-dir /opt/upp_basefix
export UPP_DIR=/opt/upp_basefix   # then build.sh finds it too (put this in ~/.bashrc)
```

Step 4: build and test

For each compiler it runs `./build.sh --compiler <gcc|clang> --test`: a **debug** build of the three programs and the core tests. The output of each goes to `out/bootstrap-gcc.txt` and `out/bootstrap-clang.txt`.

It puts **TreeSyncBuild** and **TreeSyncInstall** in the **TreeSyncApp** folder of your app list (`setup/app-folder.sh`; see 2.6).

It also points git at the project's pre-push hook (`git config core.hooksPath v001/setup/hooks`), so every `git push` first runs the full check (see 2.5).

Step 5: summary

It ends with one line per step in the start : stop : duration layout:

```
 Start    : Stop     : Duration         : Command / Comments
 14:02:58 : 14:03:01 : 000:000:00:00:03 : Check requirements........ : Result = ....0 : Good
 14:03:01 : 14:05:40 : 000:000:00:02:39 : Build and test with gcc... : Result = ....0 : Good
: passed: 290/290 checks
```

The same lines are added to `Bootstrap-Results.txt` in TreeSync's logs folder.

1.5 Bootstrap options

Option	What it does
`--dry-run`	Only say what it would do
`--yes`	Do not ask before installing (for scripts)
`--compiler gcc` / `clang` / `both`	Build with one compiler (default both)
`--no-build`	Stop after the checks and installs
`--upp-dir DIR`	Where U++ goes if it has to be fetched (default `~/upp_basefix`)
`--check`	Only check, one `@req` line per requirement (TreeSyncBuild uses this); exit 1 if a required one is missing
`-h`, `--help`	Show the help

1.6 After bootstrap

Bootstrap leaves **debug** builds in `out/gcc/` and `out/clang/`. For everyday use build the faster **release** programs, and install them:

1. **TreeSyncBuild** (TreeSyncApp folder): Type *Release*, **Build** (or `./verify.sh`: debug and release, all tests, both compilers; ends with release builds).
2. **TreeSyncInstall**: **Install GCC...** and / or **Install Clang...** copies that build to `/usr/local/bin` for root (asks for your password; or `./install.sh`). TreeSyncRoot-GCC, TreeSyncHome-GCC (and

the Clang ones) then join the TreeSyncApp folder.

Most syncs need root, and root runs the installed copy: see 2.6 TreeSyncInstall. Then go on to 3. Run.